RAPPORT TECHNIQUE

Rapport de sécurité mobile - Projet de formation

By Vanessa, Romuald, Gwendal.

Sommaire

1. Contexte et perimetre
2. Synthese des findings
3. Detail des User Stories
US 1 - Reception et verification de l'APK
US 2 - Analyse automatisee avec MobSF
US 3 - Extraction classes.dex / AndroidManifest.xml
US 4 - Decompilation avec Apktool
US 5 - Analyse de strings.xml
US 6 - Analyse du bytecode decompile
US 7 - Recompilation de l'APK
US 8 - Re-signature de l'APK
US 9 - Verification Firebase
US 11 - Politique de securite mobile
US 12 - Recommandation MDM
4. Plan de remediation
5. Conclusion

1. Contexte et périmètre

Dans le cadre de notre formation en sécurité mobile, nous avons été chargés d'auditer une application Android vulnérable : Nickel (v2.12.0, package `com.fpe.comptenickel`).

Plutôt qu'une simple vérification technique, cet exercice a été conçu comme un véritable audit de sécurité de bout en bout. Nous avons structuré notre démarche autour de 12 user stories, nous permettant de couvrir l'intégralité du cycle d'analyse, de la réception de l'APK jusqu'à l'élaboration de recommandations stratégiques.

Pour garantir la pertinence de nos résultats, notre équipe s'est appuyée sur les standards de l'OWASP Mobile Security Testing Guide (MSTG), en combinant plusieurs approches :

- Analyse statique automatisée via MobSF.
- Rétro-ingénierie et décompilation avec des outils comme apktool et jadx.
- Vérifications manuelles approfondies pour confirmer les vulnérabilités détectées.

2. Synthèse des résultats.

L'analyse statique automatisée via MobSF a attribué à l'application un score de sécurité de 49/100 (risque moyen, Grade B). Sur les 432 trackers connus par l'outil, seuls 3 ont été détectés, ce qui limite la fuite de données comportementales.

Voici la répartition des vulnérabilités identifiées par niveau de sévérité :

Severite	Nombre	Exemples
HIGH	3	MinSDK 19, trafic en clair, logging en production
MEDIUM	20	Signature Janus, SHA1, receivers exportes, permissions
INFO	2	Application signee
SECURE	2	Bonnes pratiques detectees (anti-VM, anti-debug)

L'étape la plus critique de notre audit a été la fouille du code décompilé et des fichiers de configuration. Nous avons découvert plusieurs secrets exposés en clair, ce qui constitue une faille majeure de sécurité :

Secret	Valeur (tronquee)	Risque
google_api_key	AIzaSyBTgz...	Exploitation financiere des quotas Google Cloud
firebase_database_url	application-client-nickel.firebaseio.com	Point d'entree vers la base de donnees
ACCOUNT_ENDPOINT	api.nickel.eu/customer-banking-api	Cartographie de l'infrastructure backend
PUSHWOOSH_APP ID	DDF11-D1BF9	Usurpation de notifications push (phishing)
ENVIRONMENT	production	Confirmation de l'environnement reel

3. Détail des User Stories

U 1 - Réception et vérification de l'APK

Dès la réception de l'archive, notre première priorité a été de garantir l'intégrité de la source. Nous avons procédé au calcul de l'empreinte numérique (hash SHA256) de l'APK via la commande `CertUtil`.

Le résultat obtenu concorde parfaitement avec la valeur de référence, confirmant que le fichier n'a subi aucune altération (accidentelle ou malveillante) durant le transfert.

- **SHA256 :** `f01f08c8b6e2fe4612e81bc7e3a3ba9440dad0d7a962f4e67640390dd721d528`

U 2 - Analyse automatisée avec MobSF

Nous avons soumis l'APK au framework MobSF (*Mobile Security Framework*) pour une analyse statique exhaustive. Le diagnostic global affiche un score de 49/100 (Grade B), signalant un niveau de risque modéré.

Trois vulnérabilités critiques ont particulièrement retenu notre attention :

1. **Obsolescence du système :** Le support d'Android 4.4 (`minSdkVersion 19`) expose l'app à des failles systèmes qui ne sont plus corrigées par Google.
2. **Communications non chiffrées :** L'activation de `usesCleartextTraffic=true` autorise le transit de données en HTTP, facilitant les attaques de type **Man-in-the-Middle (MitM)**.
3. **Fuite de données via les logs :** La persistance de journaux de bord (*logging*) en production risque d'exposer des informations sensibles lors de l'exécution.

Enfin, l'usage de permissions intrusives (`READ_CONTACTS`, `WRITE_EXTERNAL_STORAGE`) et de méthodes obsolètes (`USE_FINGERPRINT` au lieu de `USE_BIOMETRIC`) accentue la surface d'attaque.

U 3 - Extraction des composants clés (DEX & Manifest)

Pour approfondir l'analyse, nous avons déconstruit l'APK via la commande `unzip` afin d'isoler les composants structurants :

- `classes.dex`
- `AndroidManifest.xml`

Chaque fichier a été "hashé" (SHA256) individuellement pour assurer une traçabilité totale lors des étapes ultérieures de décompilation.

Fichier	Taille	SHA256
classes.dex	7.4 MB	57898c5978715da96560ed2692b4a8fa1a8e914b37988dcf369cc59951f6c429
AndroidManifest.xml	27 KB	80b9b101e3bf02f03eec4f2c9c9ccf4d61403612e1435949b51cec8b85d7a276

U 4 - Décompilation structurelle avec Apktool

Pour accéder aux ressources non compilées, nous avons procédé à la décompilation de l'APK via Apktool (commande : `apktool d app.apk`). Cette opération a généré un environnement de travail de 12 688 fichiers comprenant :

- Le fichier `AndroidManifest.xml` enfin lisible en clair.
- Les assets et ressources (layouts, drawables, configurations Crashlytics).
- L'intégralité du code sous forme de fichiers Smali.

L'ensemble de ces sources a été archivé et scellé par une empreinte numérique pour garantir l'intégrité de notre base d'analyse.

- SHA256 : `ea1384271fc4d57838acf9e2f172e721e9ad32572baa359da07c57287c2060b2`

U 5 - Analyse critique du fichier `strings.xml`

L'exploration des ressources textuelles (`strings.xml`) a été particulièrement fructueuse. Nous y avons débusqué cinq éléments sensibles exposés en clair, ce qui constitue une erreur de configuration majeure :

1. **Clé API Google :** Risque de détournement et de surcoût financier des services Cloud.
2. **URL Firebase :** Exposition directe du point d'entrée de la base de données.
3. **Endpoints API Bancaire :** Révélation de l'infrastructure backend (`api.nickel.eu`).
4. **ID Pushwoosh :** Possibilité d'envoyer des notifications frauduleuses (Phishing).
5. **Variable d'environnement :** Confirmation explicite du mode `production`.

U 6 - Analyse du bytecode et rétro-ingénierie (JADX)

Le passage du bytecode au Java via JADX a révélé une obfuscation avancée, la plupart des classes et méthodes sont renommées de façon générique (`a`, `b`, `zzg`), ce qui témoigne d'une volonté initiale de protéger le code.

Cependant, cette protection est rendue caduque par une faille de conception, la classe **`BuildConfig`**. Celle-ci expose "en dur" toute la configuration critique de l'écosystème Nickel :

- Cartographie complète des APIs internes (*Authentication*, *Personal Space*, etc.).
- Identifiants de services tiers (GCM, Pushwoosh).

Ce qui est recommandé : Il est impératif d'externaliser ces configurations côté serveur ou d'utiliser des mécanismes de chiffrement à l'exécution. L'accès aux APIs doit également être restreint par un contrôle strict des origines (CORS) et une authentification renforcée côté backend.

U 7 - Recompilation de l'APK

Une fois les vulnérabilités identifiées au sein du bytecode, nous avons procédé à la reconstruction de l'application via la commande `apktool b app_src`. Cette manipulation a généré un nouvel exécutable situé dans le répertoire `dist/`.

Au-delà de l'aspect technique, cette étape constitue une preuve de concept (PoC) majeure. Elle démontre qu'un attaquant capable d'extraire les sources peut non seulement analyser l'application, mais aussi y injecter du code malveillant.

U 8 - Re-signature de l'application

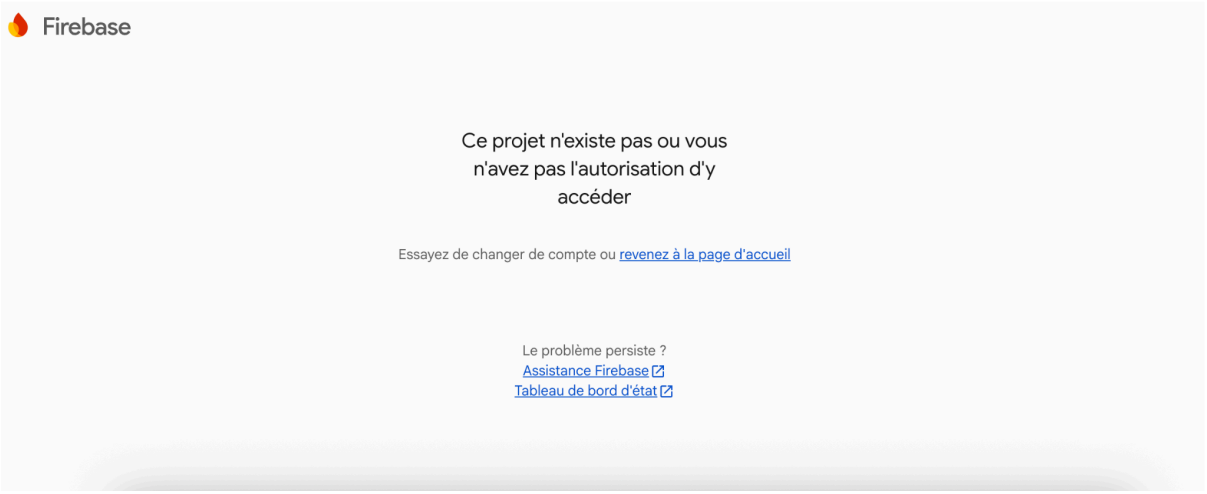
Pour qu'un APK modifié soit accepté par le système Android, il doit impérativement posséder une signature numérique valide. Nous avons donc signé l'exécutable recompilé avec un nouveau certificat généré pour l'occasion. L'outil apksigner a permis de confirmer que l'application est désormais signée selon les schémas v1, v2 et v3.

Cette phase valide le scénario d'une attaque par usurpation. Elle prouve qu'un pirate peut redistribuer une version altérée de l'application sur des stores alternatifs ou via des campagnes de phishing, tout en conservant une apparence de légitimité technique aux yeux du système d'exploitation de l'utilisateur.

U 9 - Vérification Firebase

L'URL Firebase extraite du code source (`https://application-client-nickel.firebaseio.com`) a été testée pour vérifier si la base de données était accessible publiquement. Le test a renvoyé un message d'erreur indiquant que le projet n'existe pas ou que l'accès est refusé. La base Firebase n'est donc

pas directement exploitable en l'état, ce qui constitue un point positif pour la sécurité des données.



U 11 - Politique de sécurité mobile

Une politique de sécurité mobile a été rédigée pour encadrer le développement et l'usage des applications mobiles. Elle couvre 7 mesures prioritaires :

- Verrouillage systématique des appareils (code 6 chiffres minimum ou biométrie)
- Chiffrement du disque activé sur tous les terminaux
- Séparation des usages pro/perso via conteneurisation (Work Profile)
- Mises à jour de sécurité obligatoires et automatiques
- Interdiction du root/jailbreak avec blocage automatique
- Utilisation obligatoire du VPN sur Wi-Fi public
- Capacité d'effacement à distance (Remote Wipe) configurée

U 12 - Politique de sécurité mobile

Deux solutions MDM ont été comparées pour une entreprise d'environ 300 collaborateurs : Google Workspace Endpoint Management et VMware Workspace ONE.

Critere	Google Workspace	VMware Workspace ONE
Déploiement	Très simple, rapide	Plus complexe
Integration	Excellente avec Google Workspace	Multi-environnements
Sécurité	Très bon (natif Android)	Très bon (plus avancé)
Conteneurisation	Oui (Android Enterprise)	Oui (tres avancée)
Remote wipe	Oui	Oui (granulaire)
Administration	Très simple	Plus technique
Cout	Faible a modéré	Plus élevé

Pour une structure de 300 collaborateurs utilisant deja Google Workspace, la solution Google Endpoint Management est recommandée pour son rapport simplicite/efficacite et son cout maîtrisé.